AI_CAMPUS · 온디바이스 AI · Linux & Shell

Day 1

Shell 필수 · 시스템 콜 · 프로세스

2026.10.02 (금) · 8교시 · Ubuntu 24.04 + C

```
$ ./myshell
myshell[0]$ ls /nope
ls: cannot access '/nope'
myshell[2]$ exit
```

오늘의 도착점

strace로 시스템 콜을 보고, myshell을 C로 만든다

1교시	2교시	3교시	4교시	5교시	6교시	7교시	8교시
환경 점검	**Shell 필수**	**Makefile**	**시스템 콜**	**errno · /proc**	**fork · exec**	**좀비 · 시그널**	**myshell**
check_env.sh	파이프·권한	run.sh 측정	write · strace	실패와 관찰	wait 매크로	sigaction	종합 실습

오전: C 실습을 위한 도구 준비

오후: 커널 경계를 넘는 C 프로그래밍 (C 85% · Shell 15%)

실습 폴더 ~/linux-course/day1/ · 예제 소스 src/ · 빌드 make · 모든 예제는 단독 컴파일 가능한 완결 코드

과정 지도

운영체제 과목에서 "본" 것을 C로 "일으키고 측정"한다

Day 1

시스템 콜 · 프로세스

myshell.c

Day 2

스케줄링

jitter.c

Day 3

메모리

memlab.c

Day 4

파일시스템 · sysfs

sysmon.c · mysysfs.ko

Day 5

쓰레드 · 동기화

edge_pipeline.c

최종 도착점: 수집·처리·기록 쓰레드가 링 버퍼로 데이터를 주고받는 C 프로그램에 스케줄링·CPU 고정·메모리 잠금·동기화를 적용하고, 그 효과를 /proc·/sys와 측정값으로 설명한다.

내 Ubuntu가 실습할 수 있는 상태인가

```
$ uname -r
6.8.0-45-generic
$ nproc
4
$ sudo apt install -y build-essential \
    gdb strace valgrind stress-ng \
    linux-tools-$(uname -r) \
    linux-headers-$(uname -r)
$ bash check_env.sh
```

점검 항목	왜 필요한가
코어 2개 이상	Day 2 affinity · Day 5 race
sudo 권한	RT 정책 · 모듈 로드
Secure Boot	Day 4 커널 모듈
WSL2 여부	모듈·thermal 제한

[WARN] 항목은 손을 들어 보조강사에게

이 과목의 Shell은 빌드·실행·측정 도구

이동 · 확인

```
pwd   cd ..   cd -
ls -la   cat   less
```

파일 조작

```
mkdir -p   touch   cp -r
mv   rm -r   (되돌릴 수 없음)
```

권한

```
-rwxr-xr-x
소유자 · 그룹 · 그 외
chmod +x run.sh
```

리다이렉션 · 파이프

```
> >> 2> 2>&1
ps aux | grep bash
sleep 30 &   echo $!
```

모든 프로세스는 fd 0 · 1 · 2를 가지고 태어난다

0 — **stdin**
기본 연결: 키보드

1 — **stdout**
기본 연결: 화면

2 — **stderr**
기본 연결: 화면

```
$ make > build.log 2>&1
  # 1 → build.log, 2 → 1이 가리키는 곳

$ ./a.out 2> err.txt
  # 에러만 파일로

$ grep processor /proc/cpuinfo | wc -l
  # 앞 명령의 1번 → 뒤 명령의 0번
```

순서 주의: make 2>&1 > build.log 는 에러가 여전히 화면으로 간다

바뀐 것만 다시 빌드한다

```
CC      = gcc
CFLAGS  = -Wall -Wextra -g
SRCS    = $(wildcard *.c)
TARGETS = $(SRCS:.c=)

all: $(TARGETS)

%: %.c
        $(CC) $(CFLAGS) -o $@ $<

clean:
        rm -f $(TARGETS)
```

요소	의미
`$@`	만들 대상 (03_errno)
`$<`	첫 재료 (03_errno.c)
`%: %.c`	패턴 규칙
탭	명령 줄 시작 (스페이스 X)
`mtime`	대상이 더 새로우면 건너뜀

▶ **애니메이션** D1-1 make는 무엇을 다시 빌드하나

측정은 한 번으로 믿지 않는다

```
PROG=$1
N=$2
OUT=${3:-result.csv}
echo "run,elapsed_ms" > "$OUT"
for i in $(seq 1 "$N"); do
    start=$(date +%s%N)
    "$PROG" > /dev/null
    end=$(date +%s%N)
    echo "$i,$(( (end-start)/1000000 ))" >> "$OUT"
done
```

실행

```
./run.sh ./02_syscall_direct 5 r.csv
```

결과 r.csv

```
run,elapsed_ms
1,2
2,1
3,1
4,1
5,1
```

커널에 일을 부탁하는 유일한 문



man 섹션	내용
1	사용자 명령 (ls)
2	시스템 콜 (write)
3	라이브러리 함수 (printf)

▶ **애니메이션** D1-2 사용자 모드 ↔ 커널 모드

출력 대상이 바뀌면 write 횟수와 순서가 바뀐다

터미널 `./01_write_vs_printf`

6 write 호출 수

write 1 + printf 줄마다 5

줄 단위 버퍼링

파이프 `./01_write_vs_printf | cat`

2 write 호출 수

write 1 + exit 때 fflush 1

블록 단위 버퍼링

```
$ strace -e trace=write ./01_write_vs_printf | cat
```

▶ 애니메이션 D1-3 줄 단위 vs 블록 단위

glibc 함수는 번호를 싣는 얇은 포장

```
pid_t a = getpid();            // glibc 래퍼
long  b = syscall(SYS_getpid);  // 직접

printf("SYS_getpid = %d\n", SYS_getpid);  // 39
printf("SYS_write  = %d\n", SYS_write);   // 1
```

```
$ strace -c ./02_syscall_direct > /dev/null
% time  calls  syscall
  ...      3    write
  ...      2    getpid
```

1	write
39	getpid
57	fork
59	execve
61	wait4

커널은 음수를, glibc는 -1과 errno를 준다

커널

```
return -ENOENT
rax = -2
```

→

glibc 래퍼

```
errno = 2
return -1
```

→

main()

```
fd == -1 ?
strerror(errno)
```

규칙: 반환값이 실패(-1)일 때만 errno를 읽는다

errno	이름	원인
2	ENOENT	경로 없음
13	EACCES	권한 없음
9	EBADF	닫힌 fd
21	EISDIR	디렉터리를 쓰기로

▶ **애니메이션** D1-4 실패는 어떻게 errno가 되나

읽는 순간 커널이 만들어 주는 가상 파일

1.5 MB → **52 MB**

VmRSS · malloc 전

VmRSS · 50MB malloc + memset 후

```
$ cat /proc/self/status | head
$ ls -l /proc/self/fd
$ cat /proc/loadavg
```

Day 3 예고: memset을 지우면 RSS는?

malloc은 주소만 예약한다 — 실제로 써야 페이지가 할당된다

한 번 호출, 두 번 반환

-1

부모

실패

0

자식

"나는 자식"

> 0

부모

자식의 PID

```
int x = 100;
fflush(stdout);
pid_t pid = fork();
if (pid == 0) { x += 1; exit(0); }
x -= 1;  wait(NULL);
// 자식 101, 부모 99
```

fflush를 지우고 ./05_fork_basic | cat 으로 실행하면?

stdio 버퍼도 메모리 → 자식에게 복제 → "[fork 전]" 줄이 두 번 출력된다

▶ **애니메이션** D1-5 한 번 호출, 두 번 반환

같은 PID에 다른 프로그램을 덮어쓴다

유지된다

- PID와 부모 관계(PPID)
- 열린 파일 디스크립터 0 · 1 · 2
- 현재 디렉터리, 사용자 ID

교체된다

- 코드(text) 영역
- 전역 변수 · 힙
- 스택 — 그래서 돌아올 곳이 없다

▶ **애니메이션** D1-6 같은 PID, 다른 프로그램

종료 상태를 해석하는 매크로

매크로	의미
WIFEXITED(s)	exit()로 정상 종료했나
WEXITSTATUS(s)	종료 코드 0~255
WIFSIGNALED(s)	시그널로 죽었나
WTERMSIG(s)	몇 번 시그널인가

0 — ls -l /tmp

2 — ls /nope

127 — nosuchcmd

sig 15 — sleep 30 + kill

종료 상태는 누가 가져가나

좀비 (Z)

자식은 끝났는데 부모가 wait하지 않음

```
PID   PPID STAT CMD
6021  6020 Z    [07_zombie] <defunct>
```

해결: 부모가 waitpid 호출 (kill -9는 소용없음)

고아

부모가 먼저 끝남

```
[자식] 처음 PPID=6020
[자식] 고아 후 PPID=1 (또는 subreaper)
```

init이 입양 후 수거 → 오래 남지 않는다

▶ **애니메이션** D1-7 좀비와 고아

커널이 꽂아 두는 비동기 알림

시그널	번호	언제	기본 동작
SIGINT	2	Ctrl+C	종료
SIGKILL	9	kill -9	종료 (잡을 수 없음)
SIGTERM	15	kill 기본값	종료
SIGCHLD	17	자식 종료	무시

핸들러 작성 규칙

- printf 대신 write (async-signal-safe)
- 공유 변수는 volatile sig_atomic_t
- SIGCHLD에선 waitpid(-1, ..., WNOHANG)를 반복

▶ **애니메이션** D1-8 Ctrl+C가 핸들러에 도착하기까지

bash의 정체: fork-exec-wait 루프



↺ 종료 코드를 프롬프트에 표시하고 처음으로

```
$ make myshell && ./myshell
myshell[0]$ ls /nope
myshell[2]$ sleep 100      ← Ctrl+C
myshell[130]$ exit

$ strace -f -e trace=clone,execve,wait4 ./myshell
```

왜 cd는 자식이 실행하면 안 되나

자식이 chdir("/tmp") → 자식만 /tmp로 이동 → 자식 종료 → 셸은 그대로

셸 프로세스가 직접 chdir()해야 한다 → 내장 명령

도전 과제 (선택)

1. 프롬프트에 현재 디렉터리 표시 (getcwd)
2. 끝에 & 가 있으면 백그라운드 실행 + SIGCHLD 수거
3. > 리다이렉션 (open + dup2) — Day 4 예고

오늘 정리

7개 개념, 한 줄씩

개념	핵심 한 줄
시스템 콜	사용자 프로그램이 커널에 일을 부탁하는 유일한 문
errno	실패(-1) 했을 때만 읽는 원인 번호
/proc	읽는 순간 커널이 만들어 주는 가상 파일
fork	한 번 호출, 두 번 반환 (자식 0, 부모 PID)
exec	같은 PID에 새 프로그램을 덮어씀
wait	자식의 종료 상태 수거 — 안 하면 좀비
시그널	커널이 보내는 비동기 알림, sigaction으로 처리

다음 시간 · Day 2 (10/6 화)

스케줄링

누가, 얼마나 자주, 얼마나 늦게 CPU를 받는가를 측정한다 — nice · affinity · SCHED_FIFO · jitter.c

개념 워크시트 80문항
linux-day1/anim/worksheet.html

애니메이션 8종 목록
linux-day1/anim/index.html